

NotAndOr - Informe Stage III

NotAndOr - Informe Stage III

Tabla de Contenidos

1. Introducción
2. Modelo Computacional
 1. Dominio
 2. Lenguaje
3. Implementación
 1. Frontend
 2. Backend
 3. Dificultades Encontradas
4. Futuras Extensiones
5. Conclusiones
6. Referencias
7. Bibliografía

1. Introducción

NotAndOr es un DSL para describir y simular circuitos lógicos booleanos secuenciales síncronos. El compilador recibe un programa por `stdin`, construye un AST con Flex/Bison, aplica validaciones semánticas propias del dominio y genera un simulador C99 autocontenido.

Repositorio completo: <https://github.com/tcostamendez/TPE-ATLC>

2. Modelo Computacional

2.1. Dominio

El dominio modela circuitos booleanos de tiempo discreto. Cada señal toma valores 0 o 1. Los circuitos pueden tener entradas, salidas, cables combinacionales, registros con estado e instancias de otros circuitos.

El lenguaje mantiene una separación explícita entre:

- señales sin estado, declaradas como `input`, `output` o `wire`;
- señales con estado persistente, declaradas como `reg`;

- lógica combinacional, evaluada a partir de las señales actuales;
- lógica secuencial, aplicada cuando se detecta un flanco sobre una entrada usada como reloj.

No se modelan retardos físicos, señales analógicas, buses multibit ni múltiples dominios de clock con garantías temporales. Todas las señales son booleanas.

2.2. Lenguaje

Las construcciones implementadas son:

- I. `circuit`: define un módulo reutilizable.
- II. `input`, `output`, `wire`, `reg`: declaran señales booleanas.
- III. `=`: define una asignación combinacional.
- IV. `<=`: define el próximo valor de un registro dentro de un bloque secuencial.
- V. `on rising_edge(signal)` y `on falling_edge(signal)`: definen actualizaciones ante flancos ascendentes o descendentes.
- VI. `and`, `or`, `xor`, `not`: operadores booleanos.
- VII. `.` y `+`: alias de `and` y `or`, respectivamente, para expresiones booleanas más compactas.
- VIII. `CircuitName(...) -> (...)`: instancia un subcircuito con conexiones nombradas.

`clk` no es una palabra reservada. Cualquier señal declarada como `input` puede usarse como reloj de un bloque de flanco. Esta decisión mantiene al clock como parte explícita de la interfaz del circuito y evita una semántica especial para un nombre fijo.

Código 1: registro de un bit con carga.

```
circuit Register1 {
input in, load, clk;
output out;
reg value;
wire next;

out = value;
next = (load . in) + ((not load) . value);

on rising_edge(clk) {
value <= next;
}
}
```

3. Implementación

3.1. Frontend

El frontend conserva la arquitectura Flex/Bison del proyecto base. Flex reconoce palabras reservadas, identificadores, operadores, comentarios y símbolos de puntuación. Bison consume ese stream de tokens, aplica precedencia para las expresiones booleanas y construye un AST con nodos para programas, circuitos, declaraciones, asignaciones combinacionales, bloques secuenciales, instancias y expresiones.

El AST se mantiene como frontera entre frontend y backend. Las listas se representan con nodos enlazados para acompañar el estilo del template original en C y simplificar la liberación explícita de memoria.

3.2. Backend

La primera fase del backend es el análisis semántico. Construye una tabla global de circuitos y una tabla de señales por circuito. Sobre ese modelo valida las reglas que no pueden resolverse solamente con la gramática:

- no puede haber circuitos ni señales redeclaradas;
- toda señal usada debe estar declarada;
- = no puede asignar **input** ni **reg**;
- <= solo puede asignar registros dentro de bloques secuenciales;
- un registro no puede tener múltiples definiciones secuenciales;
- una salida debe quedar determinada;
- una instancia debe referenciar un circuito existente;
- las conexiones de instancia deben respetar puertos, duplicados y cantidad de entradas/salidas;
- una salida de instancia no puede manejar una entrada ni un registro del circuito padre;
- no puede haber ciclos combinacionales;
- no puede haber instanciación recursiva de circuitos;
- la señal de flanco debe ser una entrada declarada.

Código 2: instancia válida de un subcircuito.

```
circuit AndGate {
input a, b;
output out;
out = a and b;
}

circuit Main {
input x, y;
output z;
AndGate(a=x, b=y) -> (out=z);
}
```

La fase final genera un programa C99 completo. El runtime queda incluido en el código emitido: estructuras de estado, funciones de inicialización, liberación, evaluación combinacional, avance de ciclo y `main`.

El simulador generado usa el siguiente contrato de entrada:

1. lee un entero N, la cantidad de ciclos;
2. lee N filas con los inputs del circuito top, en orden de declaración;
3. después de cada ciclo imprime los outputs del circuito top, también en orden de declaración.

Código 3: estímulo de entrada para un circuito con dos entradas.

```
4
0 0
0 1
1 0
1 1
```

La simulación de cada ciclo realiza:

1. normalización de entradas a valores booleanos;
2. evaluación de lógica combinacional;
3. detección de flancos ascendentes y descendentes;
4. cálculo de próximos valores de registros;
5. actualización de registros;
6. evaluación final de outputs.

Para evitar colisiones con palabras reservadas de C, el generador no emite nombres del DSL directamente como identificadores C. En su lugar usa prefijos internos para tipos, funciones, campos, entradas y señales previas.

Los comandos principales son:

```
docker compose run --rm compiler src/main/bash/build.sh
LOGGING_LEVEL=ERROR .build/Flex-Bison-Compiler <programa >simulator.c
LOGGING_LEVEL=ERROR .build/Flex-Bison-Compiler --top Main <programa >simulator.c
```

Si no se especifica `--top`, el compilador usa como top el último circuito definido.

La suite incluye programas aceptados, programas rechazados y fixtures de simulación. Los casos de aceptación cubren compuertas combinacionales, half-adder, flip-flops, registros, composición, precedencia, comentarios, alias simbólicos y flancos descendentes. Los casos de rechazo cubren errores léxicos, errores sintácticos y errores semánticos. Los fixtures de generación compilan el C emitido con `gcc -std=c99 -Wall -Wextra` y ejecutan el simulador comparando su salida contra archivos esperados.

3.3. Dificultades Encontradas

La migración desde el lenguaje aritmético del template hacia un DSL de circuitos obligó a reemplazar el modelo de AST y a mantener conectadas las fases existentes sin romper el flujo general del compilador. También fue necesario separar con claridad los errores sintácticos de los errores semánticos, porque muchos programas inválidos del dominio sí son parseables.

Otra dificultad fue definir una simulación suficientemente simple para la entrega pero útil para circuitos secuenciales. La solución elegida genera simuladores C99 autocontenidos y evalúa la lógica combinacional de manera iterativa, mientras que el análisis semántico rechaza ciclos combinacionales para evitar comportamientos indefinidos.

Finalmente, la validación local depende del toolchain. El host puede no tener `cmake` o puede incluir un `bison` demasiado antiguo para las directivas utilizadas. Por ese motivo, el entorno de referencia del proyecto es Docker/Ubuntu y los scripts de build y test están pensados para ejecutarse allí.

4. Futuras Extensiones

- Soporte para buses multibit.
- Inicialización explícita de registros.
- Testbenches declarados en el DSL.
- Mejor ordenamiento topológico para reducir iteraciones de estabilización combinacional.
- Reportes semánticos con ubicación exacta del AST.

5. Conclusiones

Stage III transforma el proyecto en un compilador funcional para simulación. La etapa semántica captura errores estructurales relevantes del dominio y la generación C99 produce simuladores ejecutables sin dependencias externas al compilador de C.

El resultado mantiene la arquitectura pedida por la materia, conserva Flex/Bison como frontend y completa el backend con validaciones de dominio, selección de circuito top, generación de código y fixtures de runtime.

6. Referencias

- Consigna del Proyecto Especial, 12 de marzo de 2026.
- Material de cátedra: Análisis Semántico, Generación de Código y Runtime.
- IEEE. Standard for Verilog Hardware Description Language.

7. Bibliografia

- Aho, Lam, Sethi y Ullman. Compilers: Principles, Techniques, and Tools.